

HLS4ML Cross-Compilation Workflow

For Apple Silicon (M-Series)

Peter Dunniece

April 2, 2026

1 Overview

Running Intel HLS tools (OneAPI/Quartus) on Apple Silicon creates an architecture conflict.

- **The Problem:** Intel tools require `x86_64`. macOS emulates this via Rosetta 2, but the `x86` version of TensorFlow (required for `hls4ml`) relies on AVX instructions. Rosetta 2 cannot translate AVX, causing `Illegal Instruction` crashes in a single container.
- **The Solution:** A dual-container workflow sharing a single file system.
 1. **Dev Container (ARM64):** Runs native Python/TensorFlow. Used for model training and generating C++ files.
 2. **Build Server (x86_64):** Background Intel container used only for synthesis (`cmake`, `make`).

2 Setup: Build Server

Build the Intel Docker image locally for platform consistency.

2.1 1. Dockerfile

Create a Dockerfile using the Intel OneAPI Basekit:

```
1 # Base Image: Official Intel OneAPI (x86 only)
2 FROM intel/oneapi-basekit:2025.0.0-0-devel-ubuntu22.04
3
4 # Install basic build tools and Python prerequisites
5 RUN apt-get update && apt-get install -y \
6     build-essential \
7     cmake \
8     git \
9     python3-pip \
10    && rm -rf /var/lib/apt/lists/*
11
12 # Install HLS4ML dependencies (for verification scripts)
13 RUN pip3 install numpy tensorflow hls4ml[profiling]
14
15 # Set the working directory
16 WORKDIR /workspace
```

2.2 2. Build the Image

Force the `linux/amd64` platform to trigger Rosetta 2 emulation. Run this in the folder containing your Dockerfile:

```
1 docker build --platform linux/amd64 -t hls4ml-intel-local .
```

Note: This pulls large dependencies and takes 10–20 minutes.

3 Setup: Automation Script

Create `build_report.sh` in your project root to link the development environment and the build container.

```
1 #!/bin/bash
2
3 # Default project folder name
4 PROJECT_NAME=${1:-my_project}
5
6 echo "Launching build for: $PROJECT_NAME"
7
8 # Execute build commands inside the background 'intel_builder' container
9 # CRITICAL: We source oneapi-vars.sh to load the compiler environment
10 docker exec -t intel_builder bash -c "
11     source /opt/intel/oneapi/2025.0/oneapi-vars.sh --force && \
12     cd /workspace/hls4ml-tutorial/$PROJECT_NAME && \
13     mkdir -p build && cd build && \
14     cmake .. -DTARGET_DEVICE='Intel' && \
15     make report
16 "
17
18 echo "Build complete. Check reports/report.html for results."
```

Make it executable:

```
1 chmod +x build_report.sh
```

4 Daily Workflow

4.1 1. Start the Environment (Mac Terminal)

Start the Intel container after a reboot or Docker restart.

```
1 # 1. Clean up any stopped instance first
2 docker rm -f intel_builder
3
4 # 2. Navigate to your project root (The "Parent" Folder)
5 cd "/Users/peterdunniece/Imperial Masters/Self Study Project/HLS4ML Tutorials"
6
7 # 3. Mount the current directory ($(pwd)) to /workspace inside the container
8 docker run -d -t --name intel_builder \
9     --platform linux/amd64 \
10     -v "$(pwd):/workspace" \
11     -w /workspace \
12     hls4ml-intel-local bash
```

4.2 2. Generate C++ Files (VS Code)

Run your HLS4ML script in the ARM64 Python container. Use `write()`, not `build()`.

```
1 # ... Define and Train Model ...
2
3 # Generate Project (Do not compile here)
4 hls_model = hls4ml.converters.convert_from_keras_model(
5     model,
6     hls_config=config,
7     output_dir='my_project',
8     backend='oneAPI'
9 )
10 # Write only - Compilation happens in the next step
11 hls_model.write()
12 print("C++ generated successfully.")
```

4.3 3. Synthesise (Mac Terminal)

Run the script from your Mac terminal to generate resource reports (ALMs, DSPs, Latency) in `build/reports/`.

```
1 ./build_report.sh my_project
```